LAIRS Standard
Runtime-Governed Systems

Defining the standard for continuously verified authority — and the reference implementation that meets it.

Russell Aydelotte
Aegis Secure Convergence Systems

v2.0 · July 24, 2026 · Supersedes v1.0
Public — Not Confidential — Not Proprietary

"Intelligence is local. Authority remains human. Evolution is deliberate."

Abstract

This white paper defines the **LAIRS standard for Runtime-Governed Systems** — a class of computing environment in which authority, behavior, and state changes are continuously validated through cryptographic proof, independent verification, and enforceable policy constraints *during operation*, not only at deployment. It sets out the governance model, the required architectural layers, the cryptographic and hardware foundations, the artifact specification, and the conformance criteria a system must meet to be called runtime-governed. **LAIRS** (Local Autonomous Intelligence and Reasoning System) is the reference implementation of this standard.

> ***Intelligence is local. Authority remains human. Evolution is deliberate.***

Platform Status

LAIRS is an active research and architectural platform. Since v1.0 (February 2026) it has added a hardware root of trust, post-quantum signing, and governed change-control, all now operational. It is **not yet production-commissioned** and has **not undergone external adversarial review** — a prerequisite for any production deployment. Components marked *In Development* are subject to change. No component acts autonomously or irreversibly without explicit Human Master Administrator cryptographic approval.

Nomenclature

LAIRS component names are **acronyms**, expanded here and on first use:

Acronym	Expansion
LAIRS	Local Autonomous Intelligence and Reasoning System
ANVIL	Attested Node, Vaulted Identity Lineage
SPECTER	Stateless Protocol for Ephemeral Cryptographic Token Endorsement and Revocation
LICTOR	Latching Interlock & Containment, Trusted Operational Response
GAT	Governed Action Token
GAE	Governed Adversarial Evolution
EPG	Execution Provenance Gate

Components with **descriptive names** (the name states the purpose): **PRAETOR** (Roman magistrate holding *imperium* → the sovereign reasoning seat), **Mnemos** (Mnemosyne, memory → the continuity spine), **Metatron's Gate** (the recording scribe → the authority ledger), **SentiNet** (Sentinel + Network → the verification fabric), and **VEDETTE** (a forward mounted sentry → the governed OSINT scout).

1. The Problem: Trust That Stops at the Door

Most systems establish trust once — at build time, at deployment, at login — and then assume it for the duration of operation. A signed container, a validated config, an authenticated session: each proves something *at a moment*, and nothing thereafter. Between those moments, authority drifts, provenance is lost, and there is no continuous, independent way to answer a simple question about any given action: *who authorized this, under what policy, and can that authorization be trusted here, right now?*

This gap is tolerable when software is slow and human-driven. It becomes dangerous when systems act **autonomously** — when an AI agent, an automated pipeline, or a self-managing infrastructure can take consequential action faster than a human can review it. The failure modes are structural, not incidental:

- Configuration and policy **drift** over time, unnoticed.
- **Loss of authority provenance** — an action happens, but *what* authorized it is unrecoverable.
- No way to **independently verify** that a system's current state is legitimate.
- **Weak coupling** between the layer that decides and the layer that enforces.
- Over-reliance on **perimeter or identity** — "who you are" instead of "what this specific action is permitted to do."
- **No provable governance** for automated or AI-driven changes.

Runtime governance closes the gap by making trust a **continuous, verifiable loop** rather than a one-time gate.

> ***Core principle — Trust is not assumed. Trust is continuously proven.***

2. Definition

A **Runtime-Governed System** is a computing environment in which authority, behavior, and state changes are continuously validated through cryptographic proof, independent verification, and enforceable policy constraints during operation — not only at deployment time. Unlike systems that rely on static configuration trust, a runtime-governed system maintains persistent assurance of correctness, authorization, and integrity *while executing*.

3. The Governance Model — Four Layers

The standard defines **three governance layers plus a hardware root of trust beneath them — four layers in total**. Each answers a distinct trust question at runtime, and no layer trusts the previous layer's word — each re-verifies cryptographically.

Layer	Role	Reference Component	Trust Question
0 · Trust Root	Hardware-attested signing authority; sealed keys; measured boot	**ANVIL**	*"Is the authority itself genuine and unmodified?"*
1 · Authority	Issue signed authorizations; quorum; immutable decision ledger	**Metatron's Gate**	*"Who authorized this action?"*
2 · Fabric	Independent verification; node binding; signed policy; last-known-good	**SentiNet**	*"Can this authority be trusted here?"*

Layer	Role	Reference Component	Trust Question
3 · Enforcement	Runtime enforcement; execution-provenance gating; evidence	**Sentinel Agents + EPG**	*"Is the system doing only what was authorized?"*

A defining rule of the standard: the entity that **authorizes** an action is never the entity that **executes** it, and neither holds the other's keys. In the reference implementation this is the **two-authority model** — ANVIL as signing authority and LICTOR as enforcement authority, on separate trust domains. ANVIL is deployed; **LICTOR is in development** (§10, §11), its role filled today by the enforcing components below.

4. The Governed Action Token (GAT)

Every authorized action is carried by an explicit, verifiable artifact. Without a valid one, enforcement **fails closed** — no exceptions. In LAIRS this artifact is the **Governed Action Token (GAT)** — the universal authorization primitive for any governed change, action, or enforcement:

Property	Purpose
Cryptographically signed	Tamper-evident authenticity from the issuing authority
Hardware-rooted, hybrid signature	*Target scheme* — classical **and** post-quantum (ECDSA-P256 + ML-DSA-87) at the ANVIL root; staged in observe today (§5.2)
Time bounded	Tokens expire; stale authority cannot be replayed
Context scoped	Authority limited to specific action types and contexts
Node bound	Bound to specific infrastructure nodes or identities
Independently verifiable	Third-party validation, offline, against pinned keys — no query to the issuer
Ledger anchored	Linked to an immutable decision log for provenance
Authority metadata	Records issuer, governing policy, and approval quorum
Action intent hash	Binds the token to the exact action authorized

5. Two Foundations New to This Standard

5.1 A Hardware Root of Trust

Software can sign; only hardware can attest to the *state that produced the signature*. The reference implementation, **ANVIL**, is an off-host signing appliance whose keys are sealed to a TPM and released only when the appliance boots into its expected, measured state. Every signature carries a hardware attestation quote; verifiers pin the appliance's identity, not a network address. A compromised operational host cannot forge authority, because the authority's keys were never on it.

5.2 Post-Quantum by Policy

A governance signature is a long-lived claim; a signature broken in ten years retroactively invalidates a decade of provenance. The standard therefore requires **post-quantum-aligned** signing — post-quantum cryptography (PQC) on every enforcement token, no classical-only authority. The reference cryptographic layer, **SPECTER**, uses ML-DSA-87 signatures and ML-KEM-1024 key encapsulation. Post-quantum signing is live today: under `require_pqc` the SPECTER Ghost Key emits an ML-DSA-87 signature and fails rather than degrade to a classical one. The genuine **hybrid AND-signature** — classical *and* lattice, where a forgery must break both — is produced by the ANVIL hardware root and currently runs in *observe* mode alongside the legacy signature; making it the required scheme is the pending custody cutover (§11). The **Ghost Key** ceremony assembles a single-use signing key from threshold shares and a hardware lattice seed (combined via HKDF-SHA3-512), then destroys it — the key never persists at rest.

*(A distinct quantum concern, harvest-now-decrypt-later, is a **confidentiality** threat — record ciphertext now, decrypt once a CRQC (cryptographically relevant quantum computer) exists — and is addressed not by signatures but by ML-KEM-1024 key encapsulation on material that must stay confidential.)*

6. The Audit Fabric — Proof During Execution

Runtime governance produces **decision provenance records**, not passive logs. Every governed event embeds its own governance context — actor identity, role, declared intent, active policy, approval state — and is sealed into a **segmented Merkle ledger**: events grouped into segments, each segment Merkle-rooted, hash-chained to the last, and signed by the hardware-rooted authority. Any party can verify any historical segment **offline**, with no access to the origin system, using only the segment files and the authority's public keys. Storage is **bounded** across hot/warm/cold tiers without sacrificing integrity — unlike a blockchain, there is no consensus overhead and no unbounded growth.

7. Staged Enforcement — Honest by Design

The standard mandates that every control is **proven in observe mode before it enforces**. A runtime-governed system runs a mixed, honest posture rather than a single global claim:

- **Enforcing:** the authority gate; governed, signature-re-verifying change-control; the runtime guard.
- **Observing / staged:** hardware-root-*required* verification (running alongside the legacy signature until cutover); network egress/zone *enforcement*.
- **Not yet built:** the dedicated enforcement authority (LICTOR, §10–§11) — a specified component, not a running one; its role is filled today by the enforcing components above.

In every observe stage the verification pipeline is fully active and generating evidence — the difference is only whether a failed verification *blocks* or merely *records*. This staging is what lets trust be calibrated before it is relied upon.

8. Conformance — What Makes a System Runtime-Governed

This is the rubric. A system meets the LAIRS Runtime Governance Standard if and only if it:

1. Uses cryptographically signed governance artifacts for all authorized actions.
2. Signs authority with a **post-quantum-aligned** scheme (hybrid acceptable; classical-only is non-conformant).
3. Roots signing authority in a **hardware trust anchor** that attests to its own measured state.

4. Performs **independent** runtime verification across all enforcement nodes.
5. Enforces policy through verifiable agents that **fail closed** on artifact failure.
6. Maintains immutable decision provenance via a cryptographically anchored ledger.
7. Supports safe recovery and Last-Known-Good rollback.
8. Enables governed evolution without authority loss.
9. Implements **multi-authority quorum** for high-assurance decisions.
10. **Separates signing authority from enforcement authority** across distinct trust domains.

Where LAIRS itself stands against this rubric (stated, not hidden). LAIRS meets criteria 1, 3, 4, and 6–9 today. Three are **partial**: **criterion 2** — post-quantum signing is live on the GAT / audit path, but SentiNet policy bundles are still Ed25519-only; **criterion 5** — fail-closed enforcement holds for the enforcing components (Metatron's Gate, Sentinel Forge, Praetorian Guard) while host-level Sentinel enforcement is staged; **criterion 10** — signing and enforcement are separated in practice (keys off-host on ANVIL, enforcement by separate components), but the dedicated enforcement authority, LICTOR, is not yet built. Any conformance claim for LAIRS must carry these three caveats; the full self-assessment is in the Unified Technical Standard, §12.

9. Why a Standard, Not a Product

Runtime governance is a real field with strong prior art, and honesty means naming it. Zero trust is defined by **NIST SP 800-207**. Remote-attestation semantics are standardized in **RATS (RFC 9334)**, with **Keylime** delivering TPM-based runtime attestation. Workload identity is **SPIFFE/SPIRE**. Supply-chain provenance is **in-toto**; secure, rollback-protected distribution is **TUF**. Post-quantum signing is **ML-DSA-87 (FIPS 204)**. LAIRS invents none of these — and says so.

What is largely absent is a **single conformance standard** that composes them into one continuous runtime authority loop — hardware-rooted attestation *and* per-action, PQC-signed authorization (a GAT, the provenance spirit of in-toto but for *actions* rather than build steps) *and* zero-trust re-verification at every hop *and* threshold/rollback-protected policy distribution *and* a strict separation of signing authority from enforcement authority — together with a **working reference implementation** that runs the whole loop end to end on sovereign, offline-capable infrastructure. No primitive here is novel; the *assembly*, as a measurable standard plus a running implementation, is the contribution.

That is what this document is: the category definition, and LAIRS as the reference implementation that satisfies it (see the Related Standards mapping in the Unified Technical Standard, Appendix C). A standard is not diminished by others building in the same space — it is *validated* by it.

10. Reference Implementation — Component Snapshot

Operational: LAIRS Core (cognitive runtime); ANVIL (hardware root of trust); Metatron's Gate (quorum authority, enforcing); SentiNet (verification fabric, bundle v2.2 across three nodes); Sentinel Agent (host integrity); **Sentinel Forge** (governed change-control, deployed); **EPG** (execution-provenance gate); **PRAETOR** (sovereign reasoning model — proposes governed dispositions, never disposes); **Mnemos** (persistent, charter-anchored continuity); GAE (governed adversarial evolution); Praetorian Guard; AiCraft (local training governance).

In development: LICTOR (enforcement authority); VEDETTE (governed OSINT collection); the ANVIL immutable appliance OS; Sentinel Gate; Sentinel AV; Leave No Trace.

11. Roadmap

- Complete the hardware-root custody cutover (hardware-root-required, retire the legacy classical signature).
- Bring policy-bundle trust to the GAT bar: hybrid Ed25519 × ML-DSA-87 verification.
- Build the enforcement authority (LICTOR) and two-authority action routing.
- SentiNet Phase-3 network enforcement (governed egress and zone flows).
- Ship the immutable, measured-boot, TPM-sealed appliance OS.
- Tiered multi-voter quorum; multi-peer Byzantine fault tolerance beyond three nodes.
- Formal verification of governance-protocol correctness.
- External adversarial review (cryptography, AI red-team, architecture) before commissioning.

12. Guiding Philosophy

LAIRS is designed to **empower operators, not replace them**. Authority is not delegated to the machine; it is *proven* to remain human at every step. Digital sovereignty comes from local intelligence, governed evolution, and zero trust by default — a system that earns confidence continuously rather than asserting it once.

> ***Intelligence is local. Authority remains human. Evolution is deliberate.***

References

Ref	Title	Locator
[1]	LAIRS Unified Technical Standard v2.0 — full specification and Related Standards mapping (Appendix C)	`https://github.com/ASConvergenceSystems/LAIRS_Unified_Technical_Overview`
[2]	NIST SP 800-207 — Zero Trust Architecture (2020)	`https://doi.org/10.6028/NIST.SP.800-207`
[3]	RFC 9334 — Remote ATtestation procedureS (RATS) Architecture (2023)	`https://www.rfc-editor.org/rfc/rfc9334`
[4]	FIPS 204 — Module-Lattice Digital Signature Standard, ML-DSA (2024)	`https://doi.org/10.6028/NIST.FIPS.204`
[5]	FIPS 203 — Module-Lattice Key-Encapsulation Standard, ML-KEM (2024)	`https://doi.org/10.6028/NIST.FIPS.203`
[6]	in-toto — software supply-chain integrity	`https://in-toto.io`
[7]	The Update Framework (TUF)	`https://theupdateframework.io`
[8]	SPIFFE / SPIRE — workload identity	`https://spiffe.io`
[9]	Keylime — TPM-based runtime attestation	`https://keylime.dev`